

Тестове завдання: Fullstack Developer

Стек

Node.js, React, GraphQL, PostgreSQL.

Контекст

Потрібно реалізувати систему бронювання ігрових сесій на ігрових аренах.

Користувач може:

- вибрати арену;
- вказати час початку гри;
- вказати тривалість гри;
- перевірити доступність;
- створити, редагувати або скасувати бронювання.

Кожна арена працює цілодобово, без вихідних.

Бізнес-правила

- На одній арені одночасно може проходити не більше **5 ігрових сесій**.
- Система не повинна дозволяти створити або оновити бронювання, якщо в будь-який момент часу кількість активних сесій перевищує 5.
- Сесії можуть торкатися одна одної межами — це **не вважається перетином**.
- Час початку сесії **може бути довільним** і не обов'язково кратним годині або хвилині.
- Сесія може **перетинати північ** (тобто починатися в один день, а завершуватися в інший).
- Мінімальна тривалість сесії — **5 хвилин**.
- Максимальна тривалість сесії — **24 години**.
- Якщо створення або оновлення сесії **не порушує жодного з правил**, система повинна дозволяти виконання цієї операції (без надмірних обмежень).

Приклад

- 5 сесій закінчуються о **11:00**;
- інші 5 сесій починаються о **11:00**.

Це **допустима ситуація**, оскільки сесії не перетинаються (кінець однієї дорівнює початку іншої).

Основна задача

Реалізувати UI та GraphQL backend API для керування сесіями.

Користувач повинен мати можливість:

- вибрати арену;
 - переглянути сесії для арени;
 - створити нову сесію;
 - змінити існуючу сесію;
 - видалити сесію;
 - отримати повідомлення, якщо створення або редагування неможливе.
-

Важлива вимога (конкурентність)

Система повинна бути захищена від race conditions.

Якщо два користувачі одночасно створюють сесію на одну арену — система не повинна допустити перевищення ліміту в 5 одночасних сесій.

Очікується, що кандидат продемонструє коректну роботу з транзакціями, блокуваннями або іншим надійним механізмом на рівні backend/database.

Вимоги до продуктивності

Під час тестування система працюватиме з наступним обсягом даних:

- **1000 арен;**
- для кожної арени — бронювання за **5 років інтенсивної роботи;**
- включаючи як минулі, так і майбутні сесії.
- дані будуть **штучно згенеровані** (з високою щільністю бронювань).

Важливо:

- база даних повинна мати відповідні індекси;

- запити мають виконуватись швидко;
 - рішення повинно залишатися ефективним при зростанні обсягу даних за рахунок коректного використання індексів та оптимальних запитів до бази даних.
-

Опціональна задача

Якщо сесію неможливо створити у вибраний час, система може:

- запропонувати найближчий доступний слот (слоти);
 - з урахуванням заданої тривалості.
 - пошук виконувати в межах **14 днів**;
-

Backend

Реалізувати GraphQL API з можливостями:

- отримати список арен;
 - отримати сесії по арені за задану дату;
 - перевірити доступність;
 - створити сесію;
 - оновити сесію;
 - видалити сесію.
-

Модель Session

Мінімальні поля:

- `id`;
- `arenaId`;
- `startTime`;
- `endTime` або `duration`;
- `createdAt`;
- `updatedAt`.

Можна додати додаткові поля на власний розсуд, наприклад ім'я гравця, коментар або статус.

Frontend

React UI повинен дозволяти:

- вибрати арену;
- переглянути сесії для арени за задану дату;
- створювати, редагувати та видаляти сесії;
- бачити помилки валідації;
- отримувати зрозумілий фідбек, якщо слот недоступний.

Що буде оцінюватися

- коректність логіки обмеження в 5 сесій;
- захист від race conditions;
- продуктивність і робота з індексами;
- структура backend-коду;
- якість GraphQL API;
- UX/UI;
- обробка помилок;
- валідація;
- читабельність коду.

Очікуваний результат

- репозиторій з кодом;
- інструкція для запуску;
- міграції або SQL-скрипти;
- короткий опис технічних рішень;
- (бажано) тести для ключової логіки.